

Kiwi A2A Agent Commerce Network 总体架构基线

0. 文档定位

本文是 Kiwi A2A 下一阶段的总体架构母文档。

它回答：

- Kiwi 是什么；
- 哪些能力继续继承现有实现；
- A2A、UCP、Kiwi Negotiation 分别负责什么；
- shopping-cli 在新体系中的位置；
- Buyer Agent 与 Merchant Agent 如何发现和连接；
- 商业协商对象如何进入协议；
- 身份、策略、审批、幂等、恢复和信任如何工作；
- v1.0 明确做到哪里，明确不做到哪里。

本文**不承担所有 wire-level JSON Schema 的完整定义**。

完整字段、JSON Schema、canonicalization test vectors、错误码、状态转换表等，将进入独立子规范：

Kiwi Negotiation Protocol 1.0

本文一旦基线化，其子规范不得违反本文定义的边界和安全不变量。

1. 当前实现状态

1.1 已实现并继承

Kiwi 当前已经具备并继续保留：

AgentKernel
Principal Memory
Private Vault
Main Chat Session
Isolated Task Session
Task Scheduler

Buyer:

```
search
rank
track
consultation
negotiation
selected_nonbinding

Merchant:
  catalog reading
  inventory reading
  business context
  private floor / cost vault
  write ActionCandidate

OperatorController

manual
supervised
autopilot

HardPolicy
SessionStrategy
TurnInstruction

Approval
Credential Broker

shopping-cli:
  claim
  heartbeat
  idempotency
  audit
  authoritative policy gate
```

这些能力是 Kiwi A2A 的本地自治与安全底盘，而不是历史包袱。

1.2 已设计但尚未实现

以下能力目前属于设计状态：

```
OpenClaw ACP-Runtime Adapter
Hermes ACP-Runtime Adapter
```

它们未来只是：

```
ReasoningBackend
```

不是 Kiwi 对外的 A2A 网络协议。

2. Kiwi 的重新定义

Kiwi v0.3 可以定义为：

一个持续代表 Buyer 或 Merchant、拥有长期记忆、任务能力和私有策略的 Agent-first 电商运行时。

Kiwi A2A v1.0 在此基础上进一步定义为：

一个开放的 Agent Commerce Runtime，使真实经济主体的 Buyer Agent 与 Merchant Agent 可以跨运行时、跨组织、跨平台直接发现、沟通、询价、报价、还价、澄清和形成非绑定商业共识。

Kiwi 不再围绕一个中央 Gateway 建模。

新的核心资产是：

```
Principal Agent Runtime
+
Negotiation Intelligence
+
Negotiation Protocol
+
Trust / Policy / Approval
+
Open Interoperability
```

3. 协议分层

3.1 A2A

负责：

```
Agent discovery
Agent Card
Message
Task
Artifact
contextId
streaming
```

authentication
transport

回答:

Agent 怎么和另一个 Agent 通信?

3.2 UCP

负责:

commerce capability discovery
commerce profile
catalog
cart
checkout
fulfillment
order
payment-related commerce objects

回答:

双方使用什么共同的 Commerce 语义?

3.3 Kiwi Negotiation

负责:

Inquiry
RFQ
Offer
CounterOffer
ConditionalOffer
Clarification
Withdraw
Decline
AcceptedNonbindingAgreement

回答:

成交之前，两个经济主体具体怎么谈?

Kiwi Negotiation 是 Kiwi 最重要的协议资产。

3.4 AP2 / Checkout / Payment

不进入 v1.0 Core。

预留在：

```
AcceptedNonbindingAgreement
  ↓
Transaction Handoff
  ↓
v1.1+
```

4. 设计原则

4.1 Protocol-first

支持标准 A2A 的双方允许：

```
Buyer Agent
  ⇕
Direct A2A
  ⇕
Merchant Agent
```

不要求流量经过 Kiwi Cloud。

4.2 Optional Infrastructure

Kiwi 可以未来提供：

```
Directory
Relay
Hosted Merchant
Trust Registry
Observability
Enterprise Policy Service
```

但这些不能成为协议的强制中心。

4.3 LLM 负责理解，Protocol 负责事实

自然语言：

“买 500 个的话能不能做到 80 元？
其中 300 个周五之前到。”

模型负责理解。

正式商业状态必须成为结构化对象。

4.4 Private Intent 与 Public Commitment 分离

默认 Private：

Buyer max budget
Buyer urgency
Merchant cost
Merchant floor price
Internal strategy
Private memory
Personal details

只有经过：

DisclosurePolicy
+
HardPolicy
+
Approval

允许的字段才能变成 Public Commitment。

4.5 Fail Closed

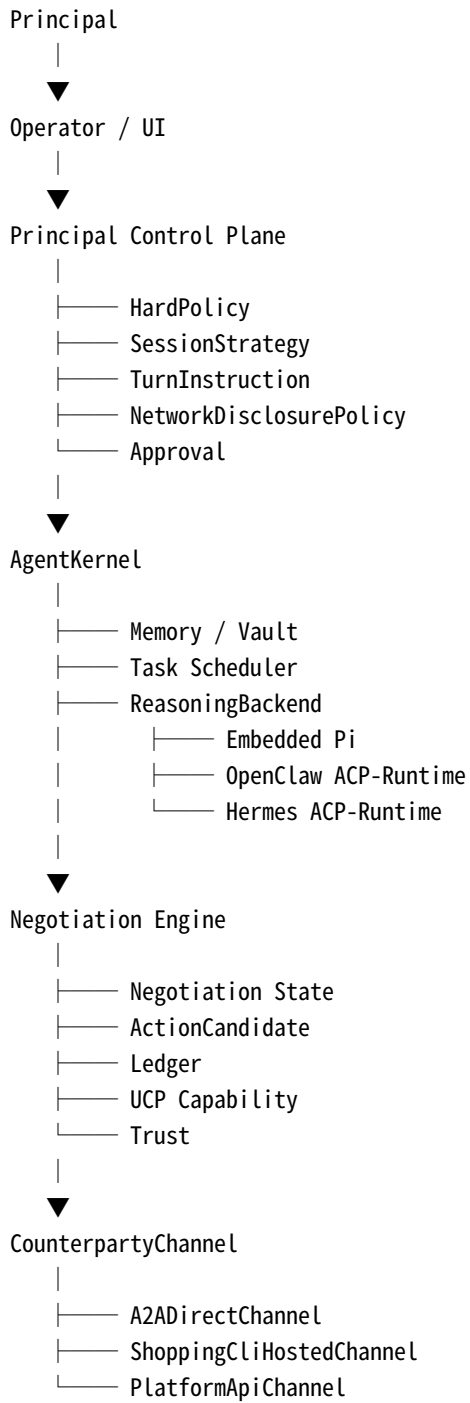
以下任何情况出现：

unknown protocol
schema invalid
identity mismatch
capability mismatch
stale approval
replay conflict

private data risk
remote state uncertainty

均不得自动产生新的商业承诺。

5. 总体架构



6. 术语迁移

旧概念	新概念	说明
CommerceConnector	CounterpartyChannel	连接的不再只是平台 API，也可能是远端 Agent
CommerceGateway	ShoppingCliHostedChannel	shopping-cli 成为一种 Channel
DecisionCandidate	deprecated compatibility term	v0.2 历史名称
ActionCandidate	ActionCandidate	统一上层候选类型
NegotiationCandidate	不再使用	上一版文档误引入
Negotiation action	NegotiationActionCandidate	ActionCandidate 的领域子类型
Marketplace Conversation	Hosted Marketplace State	hosted 路径继续存在
ACP	禁止裸用	必须写 ACP-Runtime 或 ACP-Commerce
private policy	HardPolicy / Strategy	保持已有结构
新增	NetworkDisclosurePolicy	网络披露控制

7. Candidate 类型统一

7.1 基类

```
ActionCandidate
```

表示：

Agent 建议执行，但尚未真正执行的外部有副作用动作。

7.2 Negotiation 子类型

```
NegotiationActionCandidate extends ActionCandidate
```

例如：

```
{  
  "candidate_id": "cand_...",  
}
```



```
"kind": "negotiation",
"action": "counter_offer",
"negotiation_id": "neg_...",
"expected_remote_revision": "...",
"payload": {},
"public_message": "...",
"reason_codes": [],
"risk": {},
"requires_approval": true
}
```

7.3 DecisionCandidate

`DecisionCandidate` 是 v0.2 历史接口。

迁移阶段：

```
DecisionCandidate
  ↓ adapter
NegotiationActionCandidate
```

新代码不得再创建第三套候选模型。

8. 协议资产的三个标识

Kiwi Negotiation 必须明确区分三个层次。

8.1 内部模块标识

```
kiwi.negotiation/1.0
```

仅用于：

- repo
- package
- internal logging
- schema family
- implementation version

不得直接作为公网治理 namespace。

8.2 A2A Extension URI

形式:

```
https://<kiwi-owned-domain>/a2a/extensions/negotiation/1.0
```

用途:

```
Agent Card  
A2A extension activation
```

8.3 UCP Vendor Capability

必须采用:

```
{reverse-domain}.{service}.{capability}
```

最终示例取决于 Kiwi 实际控制域名, 例如:

```
com.example.shopping.negotiation
```

其:

```
spec  
schema
```

必须托管在该 namespace 所代表的 domain authority 下。

8.4 三者关系

```
internal:  
kiwi.negotiation/1.0  
  
    | implements  
    ▼  
  
A2A:  
https://kiwi.example/a2a/extensions/negotiation/1.0  
  
    | exposes commerce capability
```



UCP:
example.kiwi.shopping.negotiation

三者不是同一个字符串，也不得互换。

9. Identifier Model

这是 v1.0 的核心协议数据模型。

9.1 negotiation_id

Kiwi 本地生成。

作用：

标识一个业务意义上的完整谈判。

示例：

neg_01...

特点：

- 跨进程稳定；
- 一次谈判只有一个；
- 可以对应多个 A2A Task；
- 可以对应多个 Message；
- 可以对应一个 A2A contextId。

9.2 A2A contextId

由 A2A 会话语义管理。

Kiwi 保存：

negotiation_id
↔
remote contextId

Kiwi 不推断其内部结构。

9.3 exchange_id

标识一个语义交换。

例如：

```
RFQ → Offer  
Offer → CounterOffer  
CounterOffer → ConditionalOffer
```

一个 negotiation 可以包含多个 exchange。

9.4 message_id

每个 Kiwi Negotiation wire message 唯一。

生成方：

```
sender
```

作用：

```
idempotency  
replay detection  
ledger reference
```

9.5 taskId

只在协商需要 A2A Task 时存在。

例如：

```
等待人工报价  
供应链核价  
企业审批  
异步库存确认
```

一个 negotiation 可存在多个 taskId。

9.6 offer_id

每个 Offer / ConditionalOffer 独立拥有。

CounterOffer 必须引用：

```
responding_to_offer_id
```

Agreement 必须引用最终 agreed offer/counter chain。

10. 基础数据类型

10.1 Money

全协议只能存在一种金钱表示：

```
{  
  "currency": "CNY",  
  "amount_minor": 83500  
}
```

禁止：

```
835  
83500  
"835元"
```

作为协议级价格值。

自然语言可以存在于 `public_message` 。

10.2 Quantity

```
{  
  "value": 200,  
  "unit": "piece"  
}
```

10.3 TermSet

所有 Offer 类对象共享 TermSet:

```
{
  "items": [],
  "price_terms": {},
  "fulfillment_terms": {},
  "service_terms": {},
  "payment_terms": {},
  "valid_until": "...
}
```

v1.0 中 `payment_terms` 只能表达谈判条件，不能执行支付。

11. Negotiation Objects

11.1 Inquiry

一般咨询。

```
{
  "type": "inquiry",
  "subject": {
    "sku": "SKU-001"
  },
  "questions": [
    {
      "code": "delivery.estimated_date"
    }
  ]
}
```

11.2 RFQ

```
{
  "type": "rfq",
  "items": [
    {
      "sku": "SKU-001",
      "quantity": {
        "value": 200,
        "unit": "piece"
      }
    }
  ]
}
```

```
    }
  }
],
"requested_terms": {
  "delivery_before": "2026-08-20T18:00:00Z"
}
}
```

11.3 Offer

```
{
  "type": "offer",
  "offer_id": "off_...",
  "terms": {
    "items": [
      {
        "sku": "SKU-001",
        "quantity": {
          "value": 200,
          "unit": "piece"
        },
        "unit_price": {
          "currency": "CNY",
          "amount_minor": 85000
        }
      }
    ],
    "fulfillment_terms": {
      "delivery_before": "2026-08-20T18:00:00Z"
    },
    "valid_until": "2026-08-06T12:00:00Z"
  }
}
```

11.4 CounterOffer

CounterOffer 不能只发送差异而完全失去上下文。

它至少包含：

```
{
  "type": "counter_offer",
  "responding_to_offer_id": "off_...",
  "proposed_terms": {
    "items": [
```

```

{
  "sku": "SKU-001",
  "quantity": {
    "value": 200,
    "unit": "piece"
  },
  "unit_price": {
    "currency": "CNY",
    "amount_minor": 83500
  }
}
]
}
}

```

完整子规范再决定：

full replacement

还是：

typed patch

v1.0 首选完整 `proposed_terms`，避免 patch 语义歧义。

12. ConditionalOffer

ConditionalOffer 是 Kiwi Negotiation 的关键对象。

不能允许任意脚本或表达式。

12.1 基本结构

```

{
  "type": "conditional_offer",
  "offer_id": "off_...",
  "base_terms": {},
  "conditions": [
    {
      "when": {
        "all": [
          {
            "field": "items.quantity",
            "op": "gte",
            "value": 500

```



```
    }  
  ]  
},  
"then": {  
  "unit_price": {  
    "currency": "CNY",  
    "amount_minor": 8000  
  }  
}  
}  
]  
}
```

12.2 支持的逻辑

v1.0 只支持：

```
all  
any
```

最大逻辑深度：

```
2
```

禁止无限嵌套。

12.3 支持的比较符

初始集合：

```
eq  
neq  
gt  
gte  
lt  
lte  
in
```

禁止：

```
eval  
regex executable expression  
javascript
```

```
python
SQL
arbitrary expression
```

12.4 Field Vocabulary

`field` 不能是任意 JSONPath。

必须来自协议定义的 allowlist，例如：

```
items.quantity
fulfillment.batch_count
fulfillment.delivery_before
buyer.segment
service.warranty_months
```

这样才能确保：

```
deterministic evaluation
+
cross-agent interoperability
```

12.5 多个 Condition

多个 `conditions[]` 默认：

```
独立 rule
```

若同时命中，必须确保结果无冲突。

发生冲突：

```
condition_conflict
```

不得由 LLM 自行选择哪个生效。

13. Clarification

```
{
  "type": "clarification",
  "questions": [
    {
      "field": "fulfillment.delivery_before",
      "reason": "missing"
    }
  ]
}
```

Clarification 可以由 Buyer 或 Merchant 发起。

14. Withdraw / Decline

14.1 Withdraw

用于撤回：

```
RFQ
Offer
CounterOffer
```

例如：

```
{
  "type": "withdraw",
  "target_id": "off_...",
  "reason_code": "commercial_terms_changed"
}
```

14.2 Decline

```
{
  "type": "decline",
  "target_id": "off_...",
  "reason_code": "terms_unacceptable"
}
```

`reason_detail` 可选，且必须经过 DisclosurePolicy。

15. AcceptedNonbindingAgreement

这是 Kiwi v1.0 的协议终点。

```
{
  "type": "accepted_nonbinding_agreement",
  "agreement_id": "agr_...",
  "negotiation_id": "neg_...",
  "agreed_offer_id": "off_...",
  "agreed_terms": {
    "items": [],
    "fulfillment_terms": {},
    "service_terms": {}
  },
  "accepted_by": [
    "buyer",
    "merchant"
  ],
  "created_at": "...",
  "binding_effect": "nonbinding",
  "creates_order": false,
  "reserves_inventory": false,
  "authorizes_payment": false
}
```

三个布尔字段不是装饰。

它们用于防止 Agent 或上层应用把 Agreement 错误解释为：

```
Order
Reservation
PaymentAuthorization
```

16. selected_nonbinding

`selected_nonbinding` 不是 A2A 协议状态。

它属于：

```
Buyer Task State
```

场景：

Buyer 同时与三个 Merchant 达成：

```
Agreement A
Agreement B
Agreement C
```

用户选择 B。

于是：

```
buyer_task.selected_nonbinding
    =
Agreement B
```

Merchant 无需知道 Buyer 是否选择了其他 Merchant。

17. Wire Envelope

对外 Envelope 使用公网 namespace，而不是内部模块名。

示意：

```
{
  "protocol": "com.example.shopping.negotiation",
  "protocol_version": "1.0",
  "negotiation_id": "neg_...",
  "exchange_id": "ex_...",
  "message_id": "msg_...",
  "in_reply_to": "msg_...",
  "actor": "buyer",
  "action": "counter_offer",
  "created_at": "...",
  "payload": {},
  "public_message": "...",
  "digest": "sha256:..."
}
```

`actor` 枚举：

```
buyer
merchant
```

system 不得伪装成交易主体。

系统事件进入 Ledger Event，而不是 Commerce Envelope。

18. Digest 与幂等

18.1 Canonicalization

Kiwi Negotiation JSON 必须有唯一 canonical representation。

v1.0 使用：

RFC 8785 JSON Canonicalization Scheme

18.2 Digest

计算：

```
canonical(  
  envelope excluding:  
    digest  
    transport-specific signature  
)  
  ↓  
SHA-256
```

得到：

digest

18.3 Idempotency Key

协议级幂等主键：

(sender_identity, message_id)

18.4 Duplicate Same Payload

若：

```
same sender
same message_id
same digest
```

receiver 必须:

```
不重新执行
返回原结果或等价 acknowledgment
```

18.5 Duplicate Different Payload

若:

```
same sender
same message_id
different digest
```

返回:

```
idempotency_conflict
```

并:

```
fail closed
```

18.6 Retention

Idempotency index:

```
至少覆盖:
max(
  offer validity,
  task lifetime,
  24 hours
)
```

Negotiation Ledger 生命周期由用户和企业 retention policy 控制。

19. Message 与 A2A Task 的分界

Message

适用于：

可以在当前 Agent turn 内完成
不需要人工后台处理
不需要长时间等待外部系统

例如：

Inquiry
simple RFQ
Offer
CounterOffer
Clarification

Task

满足任一条件则使用 Task：

需要 human approval
需要供应链核价
需要企业内部审批
需要等待库存
预计不是当前 bounded turn 可完成
需要异步 Artifact

示例：

RFQ
↓
Task working
↓
Merchant internal pricing
↓
Offer Artifact
↓
Task completed

业务对象仍然是：

Offer

Task 只是其异步生命周期载体。

20. 状态模型拆分

上一版把：

Business phase
Message type
Human approval
A2A task

混在一张状态机里。

v1.1 基线拆成三个正交状态机。

20.1 Negotiation Phase

OPEN
AWAITING_CLARIFICATION
OFFER_OPEN
AGREEMENT_REACHED
DECLINED
WITHDRAWN
EXPIRED
CANCELLED

事件驱动状态转换，例如：

RFQ received
→ OPEN

Clarification requested
→ AWAITING_CLARIFICATION

Offer created
→ OFFER_OPEN

CounterOffer
→ OFFER_OPEN

Offer expired

→ EXPIRED

AcceptedNonbindingAgreement

→ AGREEMENT_REACHED

20.2 Approval State

NOT_REQUIRED

PENDING

APPROVED

REJECTED

STALE

任何 Negotiation Phase 都可能同时存在：

PENDING approval

所以 `HumanRequired` 不再伪装成 Negotiation 状态。

20.3 A2A Task State

完全遵守 A2A Task 生命周期。

Kiwi 不重新发明另一套 Task state。

21. Approval Pipeline

新统一流程：

```
ReasoningBackend
  ↓
NegotiationActionCandidate
  ↓
bind current local + remote context
  ↓
schema validation
  ↓
HardPolicy
  ↓
NetworkDisclosurePolicy
  ↓
counterparty capability check
```

```
↓
risk assessment
↓
Approval Gate
↓
approved
↓
RE-READ REMOTE STATE
↓
RE-VALIDATE PRECONDITIONS
↓
send
```

这是对旧 v0.2 流程的有意升级。

21.1 Approval Staleness

批准必须绑定：

```
candidate_id
candidate_digest
remote_revision
policy_version
```

任意发生变化：

```
payload changed
remote offer changed
offer expired
policy changed
identity changed
```

Approval 自动进入：

```
STALE
```

不得发送。

22. State Domains

v1.0 明确八个状态域。

1. User Chat Session

2. Principal Memory
3. Private Vault
4. Task State
5. Operator Control State
6. Reasoning Session
7. Negotiation Ledger / Direct Remote Context
8. Hosted Marketplace Conversation

第 8 域只存在于：

```
ShoppingCliHostedChannel
```

Direct A2A 不假装存在一个中央 Marketplace DB。

23. Authority Model

Hosted

```
shopping-cli authoritative snapshot
>
local cache
>
reasoning state
```

Direct A2A

权威商业证据来自：

```
remote protocol response
+
confirmed local Ledger entry
```

LLM transcript 永远不是 authoritative state。

24. Negotiation Ledger

Ledger 是：

append-only
content-addressed
auditable

v1.0 建议进一步使用：

hash-linked events

每条记录包含：

event_digest
previous_event_digest

所以：

verifyChain()

有明确含义。

24.1 Ledger 保存

保存：

wire payload
message_id
exchange_id
remote contextId
remote taskId
digest
acknowledgment
identity snapshot
capability snapshot
state transitions
retries
errors

不保存：

raw chain-of-thought
private vault plaintext
irrelevant principal memory

25. Recovery

恢复不能只回放本地 Ledger。

标准恢复流程：

1. Load local negotiation
2. Load Ledger high-water mark
3. Resolve CounterpartyChannel
4. Re-fetch remote context / task state
5. Compare remote acknowledged messages
6. Reconcile remote state × Ledger
7. Expire stale candidates / approvals
8. Resume scheduler

25.1 Remote Ahead

远端出现本地没有的已确认事件：

```
fetch
validate
append Ledger
```

25.2 Local Pending

本地有 outbound message，但无法确认是否被远端接受：

如果：

```
same message_id
same digest
```

允许通过幂等机制安全重放。

25.3 Conflict

出现：

```
same message ID / different digest
unknown remote revision
```

```
identity changed
incompatible state
```

进入：

```
reconciliation_required
```

默认：

```
human review
```

26. Hosted 与 Direct 的可靠性模型

26.1 Hosted

继续使用：

```
claim
heartbeat
complete
fail
abandon
stale recovery
```

这些是：

```
ShoppingCliHostedChannel
```

内部机制。

26.2 Direct A2A

不使用伪造的 claim/heartbeat。

使用：

```
A2A Message
A2A Task
Subscribe / Poll
message idempotency
```

Negotiation Ledger
reconciliation

27. UCP Integration

Merchant 通过:

```
/.well-known/ucp
```

发布 UCP Profile。

支持 A2A 时:

```
{  
  "transport": "a2a",  
  "endpoint": "https://merchant.example/.well-known/agent-card.json"  
}
```

A2A endpoint 指向 Agent Card。

27.1 Buyer / Platform Profile Advertisement

Buyer-side Kiwi 同样可以发布自己的 UCP Profile。

A2A-over-HTTP 请求必须携带:

```
UCP-Agent
```

标识自身 profile URI。

这使 Merchant 能:

```
fetch Buyer profile  
→ validate identity  
→ intersect capabilities
```

27.2 Negotiation Capability

Kiwi Negotiation 默认定义为:

Vendor Root Capability

不是 UCP extension。

即：

不带 extends

除非未来 Kiwi 明确扩展一个标准 UCP parent capability。

28. Agent Card

Agent Card 至少需要描述：

```
name
description
provider
version
supportedInterfaces
securitySchemes
security
capabilities
skills
extensions
```

示意：

```
{
  "name": "Example Merchant Agent",
  "description": "Merchant commerce negotiation agent",
  "version": "1.0.0",
  "supportedInterfaces": [
    {
      "url": "https://merchant.example/a2a",
      "protocolBinding": "JSONRPC",
      "protocolVersion": "1.0"
    }
  ],
  "capabilities": {
    "extendedAgentCard": true,
    "extensions": [
      {
        "uri": "https://kiwi.example/a2a/extensions/negotiation/1.0",
        "required": false
      }
    ]
  }
}
```

```
]
}
}
```

Kiwi 不规定所有 Agent 必须使用 JSONRPC。

根据双方能力可选择：

```
JSONRPC
GRPC
HTTP+JSON
or future compliant bindings
```

29. Trust Model

Trust 拆成三个维度。

29.1 Identity Trust

回答：

你是谁？

来源：

```
HTTPS domain
OAuth
OIDC
mTLS
UCP profile signing key
HTTP Message Signature
Agent Card JWS
```

29.2 Protocol Trust

回答：

你是否正确遵守协议？

例如：

schema validity
replay behavior
capability accuracy
timeout behavior
signature validity

29.3 Merchant Reputation

回答：

过去与你做生意是否靠谱？

来源可以是：

local Ledger experience
external reputation provider
platform reputation
user feedback

如果没有 reputation：

unknown

不能偷偷当成：

neutral 0.5

Ranker 必须明确处理 missing value。

30. Trust Levels

建议初始四级。

T0 UNKNOWN
T1 DISCOVERED
T2 AUTHENTICATED
T3 VERIFIED_RELATIONSHIP

例如：

T0

只允许：

profile inspection

T1

可以：

low-risk inquiry

T2

可以：

RFQ
Offer
CounterOffer

T3

在用户策略允许时：

higher automation

高价值交易仍可要求人工审批。

31. Message Signature

digest 不是身份认证。

Kiwi 必须区分：

Integrity
Authentication
Non-repudiation evidence

UCP 场景优先复用：

```
HTTP Message Signatures
+
profile signing_keys
```

A2A Agent Card 可以验证签名。

是否强制 Agent Card JWS:

```
由 TrustPolicy 决定
```

而不是所有部署一刀切。

32. NetworkDisclosurePolicy

新增策略层:

```
HardPolicy
SessionStrategy
TurnInstruction
NetworkDisclosurePolicy
```

控制:

```
location precision
organization identity
buyer urgency
contact information
purchase quantity
budget hints
customer segment
historical preferences
```

33. RFQ Fan-out Privacy

Buyer 不能默认把完整需求广播给所有 Merchant。

Fan-out 必须受策略控制:

```
max_recipients
minimum_trust
disclosure_profile
```

anonymous_first_round
category sensitivity

例如：

第一轮：
5 merchants
只披露 SKU + quantity range

第二轮：
Top 2
披露精确 quantity + delivery requirements

34. Abuse Mitigation

Merchant Agent 必须防：

RFQ spam
price scraping
resource exhaustion
identity cycling
malformed schema floods
replay floods
capability probing

至少具备：

rate limiting
per-identity quota
per-domain quota
backoff
request budget
max task concurrency
payload size limit
trust-based throttling

35. Error Model

Kiwi Negotiation 定义领域错误。

第一版至少包含：

```
protocol_version_unsupported
capability_incompatible
schema_invalid
field_unsupported
identity_rejected
authentication_required
authorization_failed
offer_expired
offer_withdrawn
condition_conflict
state_conflict
approval_required
idempotency_conflict
replay_detected
rate_limited
temporarily_unavailable
```

错误不是：

```
Decline
```

两者必须区分。

`Decline` 是商业决定。

`schema_invalid` 是协议错误。

36. AgentDiscovery 与 CounterpartyChannel 职责

上一版两个接口发生职责重叠。

新模型明确：

AgentDiscovery

负责：

```
domain → UCP Profile
profile → Agent Card
capability intersection
identity bootstrap
channel candidates
```

接口：

```
interface AgentDiscovery {  
  resolve(input: DiscoveryInput): Promise<CounterpartyProfile>;  
}
```

CounterpartyChannel

不再负责 discovery。

负责：

```
open  
send  
receive  
state  
subscribe  
close
```

接口：

```
interface CounterpartyChannel {  
  open(input: ChannelOpenInput): Promise<RemoteContext>;  
  send(message: NegotiationEnvelope): Promise<ChannelResult>;  
  getState(ref: RemoteRef): Promise<RemoteState>;  
  subscribe?(ref: RemoteRef): AsyncIterable<RemoteEvent>;  
  close?(ref: RemoteRef): Promise<void>;  
}
```

37. shopping-cli 的最终定位

shopping-cli 不删除。

从：

Kiwi 唯一 Commerce Gateway

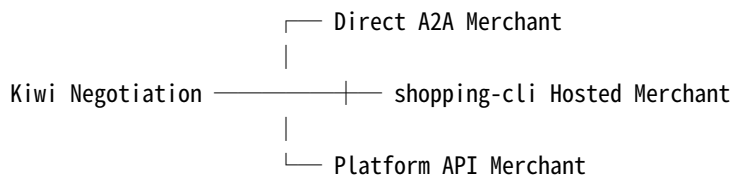
变为：

Kiwi Hosted / Legacy Commerce Infrastructure

支持：

Legacy Merchant
Hosted Merchant
queue
claim
heartbeat
audit
policy gate
old negotiation contract

架构:



38. Legacy Protocol Migration

现有:

shopping.negotiation/0.1

继续冻结。

新 canonical domain:

kiwi.negotiation/1.0

通过:

LegacyNegotiationAdapter

转换。

规则:

lossless → convert
lossy → fail closed
unsupported → human / fallback

绝不能为了兼容旧协议偷偷丢弃：

conditions
expiry
identity
agreement semantics

39. Security Invariants

标记：

[E] existing enforcement
[N] v1.0 new invariant

1. [E] 一个 Kiwi 实例只代表一个 Principal Role。
2. [E] Buyer 与 Merchant Memory/Vault/Credentials 隔离。
3. [E] ReasoningBackend 不拥有 Commerce 写权限。
4. [E] LLM 输出不直接执行。
5. [E] 不保存 raw chain-of-thought。
6. [E] v1.0 不创建订单。
7. [E] v1.0 不支付。
8. [E] v1.0 不退款。
9. [E] v1.0 不锁库存。
10. [N] remote Agent 不获得 Principal Memory。
11. [N] public message 不得绕过 structured payload。
12. [N] Direct A2A 写入必须有幂等语义。
13. [N] approval 必须绑定 remote revision。
14. [N] remote state 变化使 approval stale。
15. [N] unknown protocol/version fail closed。
16. [N] identity mismatch fail closed。
17. [N] duplicate ID + different digest fail closed。
18. [N] Direct Channel 失败不得自动降级到权限更宽 Channel。
19. [N] Legacy Adapter 不得扩大权限。
20. [N] Condition evaluation 不允许 executable expressions。

40. 可靠性前置整改

在 Direct A2A 开发之前，必须先完成 Hosted Runtime 已知问题。

P0：

```
claim escape recovery
fake claim semantics
```

P1:

```
--once signal cleanup
log redaction
filesystem permissions
```

这些不是 A2A 特有问題，但会成为多 Channel Runtime 的可靠性基础。

41. 推荐代码结构

```
src/
├── agent/
├── operator/
├── reasoning/
│   ├── embedded-pi/
│   ├── openclaw-acp-runtime/
│   └── hermes-acp-runtime/
├── discovery/
│   ├── ucp/
│   ├── agent-card/
│   └── capability/
├── a2a/
│   ├── client/
│   ├── server/
│   ├── auth/
│   └── extensions/
├── negotiation/
│   ├── domain/
│   ├── schema/
│   ├── candidate/
│   ├── engine/
│   ├── condition/
│   ├── state/
│   ├── ledger/
│   ├── idempotency/
│   ├── recovery/
│   └── errors/
├── counterparty/
└── channel.ts
```

```

|   |—— a2a-direct/
|   |—— shopping-cli-hosted/
|   |—— platform-api/
|
|—— trust/
|   |—— identity/
|   |—— protocol/
|   |—— reputation/
|
|—— protocol/
|   |—— kiwi-negotiation/
|   |—— legacy-shopping-negotiation/

```

42. Roadmap

v0.3

现有 Agent-first Runtime。

v0.4 — Protocol Foundation

首先完成：

```

reliability fixes
terminology migration
ActionCandidate unification
Negotiation domain objects
Identifier model
Condition evaluator
Ledger
Idempotency
Legacy Adapter

```

用户体验基本不改变。

v0.5 — Native A2A

完成：

```

A2A Client
A2A Server
Agent Card
Direct Channel

```

Message
Task
context recovery
authentication

v0.6 — UCP Interop

完成：

/.well-known/ucp
UCP profile
a2a service binding
profile advertisement
capability intersection
Kiwi vendor capability
spec/schema hosting

v0.7 — Open Network

完成：

multi-merchant RFQ
Trust
fan-out policy
rate limiting
abuse mitigation
interoperability tests
optional directory

Kiwi A2A v1.0

满足：

Buyer Kiwi
↕
Open A2A
↕
Merchant Agent

并完整完成：

Need

- Discovery
- Inquiry / RFQ
- Offer
- CounterOffer
- ConditionalOffer
- Clarification
- AcceptedNonbindingAgreement

43. v1.0 完成定义

Kiwi 只有同时满足以下条件，才宣布 A2A v1.0:

1. Buyer 和 Merchant 都能作为独立 A2A Agent。
2. 不依赖共同 Kiwi Gateway 即可完成谈判。
3. 可以发现 UCP Profile。
4. 可以发现 Agent Card。
5. 可以协商双方 capability intersection。
6. Kiwi Negotiation 有公开稳定 namespace。
7. 七类核心 Negotiation Objects 有冻结 Schema。
8. ConditionalOffer 可以确定性求值。
9. ID 模型有明确生命周期。
10. Message 重放具有幂等性。
11. duplicate ID / different digest 可检测。
12. 多轮谈判可跨进程恢复。
13. A2A Task 可跨进程恢复。
14. remote/local divergence 可 reconciliation。
15. approval stale 可检测。
16. Private Memory 不进入 remote context。
17. Hosted 与 Direct 共用同一 domain state semantics。
18. Legacy Adapter 可保持旧 shopping-cli 兼容。
19. Pi / OpenClaw / Hermes 不改变 wire semantics。
20. 安全测试全部通过。
21. RFQ abuse tests 全部通过。
22. network partition/replay/restart tests 全部通过。
23. 没有订单副作用。
24. 没有支付副作用。
25. 没有库存预留副作用。

44. 下一份规范

本母文档基线化以后，立即拆出：

```
docs/protocol/  
  kiwi-negotiation-protocol-1.0.md  
schemas/  
  envelope.schema.json  
  inquiry.schema.json  
  rfq.schema.json  
  offer.schema.json  
  counter-offer.schema.json  
  conditional-offer.schema.json  
  clarification.schema.json  
  withdraw.schema.json  
  decline.schema.json  
  agreement.schema.json
```

该子规范必须包含：

```
normative MUST / SHOULD / MAY  
JSON Schema  
ID generation  
canonicalization  
digest  
idempotency  
condition grammar  
state transition table  
error codes  
A2A binding  
UCP advertisement  
test vectors  
interoperability cases
```

45. Kiwi 的核心壁垒

Kiwi 不应该把壁垒建立在：

```
调用某一个大模型
```

也不应该只建立在：

```
一个中心 Shopping Gateway
```

更长期的壁垒应该是：

长期代表 Principal 的 Agent
+
Private Preference Model
+
Negotiation Intelligence
+
Negotiation Protocol
+
Trust / Policy
+
A2A / UCP Interoperability

最终 Kiwi 所代表的不是：

一个“会买东西的聊天机器人”。

而是：

一个能够长期代表真实经济主体，在开放 Agent 网络中寻找交易对手、保护私有意图、协商商业条件、形成可验证共识，并最终把共识安全交给交易系统的经济 Agent。